Université Mohammed V de Rabat
École Nationale Supérieure
d'Informatique et d'Analyse des Systèmes

جامعة محمد الخامس بالرباط
Université Mohammed V de Rabat

Deep Learning – Lecture 3

Rétropropagation et différentiation automatique

Pr. Aissam BERRAHOU

ENSIAS – Université Mohammed V de Rabat

Module Deep Learning – 2026-2027

Objectifs de la lecture

- décrire le cycle d'apprentissage : forward, coût, backward, mise à jour ;
- dériver les formules de la rétropropagation d'un MLP ;
- dérouler à la main un pas complet sur un petit réseau ;
- représenter un calcul par un graphe et y propager les gradients ;
- distinguer différences finies, dérivation symbolique et différentiation automatique ;
- utiliser `tf.GradientTape` et vérifier un gradient numériquement.

Plan de la lecture

1. Le cycle d'apprentissage

2. L'algorithme de rétropropagation

3. Graphes de calcul

4. Différentiation automatique

Plan

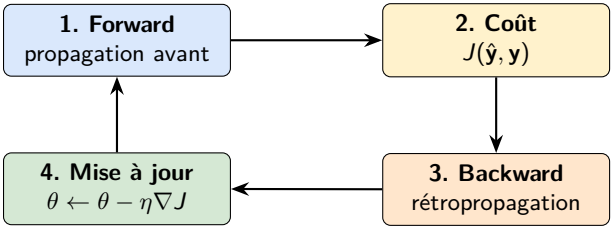
1. Le cycle d'apprentissage

2. L'algorithme de rétropropagation

3. Graphes de calcul

4. Différentiation automatique

Quatre phases



Époque, lot, itération

$n = 60\,000$, lots de $B = 128 \Rightarrow 469$ itérations (forward + backward + mise à jour) par époque.

Plan

1. Le cycle d'apprentissage

2. L'algorithme de rétropropagation

3. Graphes de calcul

4. Différentiation automatique

Pourquoi la rétropropagation ?

- Les sorties attendues des neurones cachés sont inconnues.
- L'erreur dépend de tous les poids, non linéairement.
- Il faut $\partial J/\partial w$ pour **des millions** de poids.

Idée

Appliquer la règle de dérivation en chaîne de la sortie vers l'entrée, en **réutilisant** les termes intermédiaires : un seul passage arrière.

Erreur locale

$$\boldsymbol{\delta}^{(l)} = \frac{\partial J}{\partial \mathbf{z}^{(l)}}$$

Les formules

Sortie (Softmax/sigmoïde + entropie croisée)

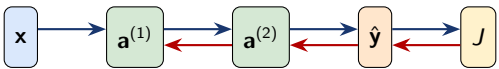
$$\boldsymbol{\delta}^{(L)} = \hat{\mathbf{y}} - \mathbf{y}$$

Couches cachées

$$\boldsymbol{\delta}^{(l)} = \left((W^{(l+1)})^T \boldsymbol{\delta}^{(l+1)}\right) \odot g'^{(l)}(\mathbf{z}^{(l)})$$

Gradients des paramètres

$$\frac{\partial J}{\partial W^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{a}^{(l-1)})^T, \qquad \frac{\partial J}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)}$$



Exemple complet : réseau 2–2–1

$\mathbf{x} = (1,0)^T$, $y = 1$, $\eta = 0{,}1$, $W^{(1)} = \begin{bmatrix} 0{,}5 & -0{,}3 \\ 0{,}2 & 0{,}4 \end{bmatrix}$, $\mathbf{b}^{(1)} = \begin{bmatrix} 0{,}1 \\ -0{,}2 \end{bmatrix}$, $W^{(2)} = [0{,}6 \quad -0{,}1]$, $b^{(2)} = 0{,}3$

Forward et coût

$\mathbf{z}^{(1)} = (0{,}6;\ 0)$, $\mathbf{a}^{(1)} = (0{,}6;\ 0)$ (ReLU)
$z^{(2)} = 0{,}66$, $\hat{y} = \sigma(0{,}66) = 0{,}659$
$J = -\ln 0{,}659 = 0{,}417$

Backward

$\delta^{(2)} = 0{,}659 - 1 = -0{,}341$
$\boldsymbol{\delta}^{(1)} = (0{,}6;\ -0{,}1)\delta^{(2)} \odot (1;0) = (-0{,}205;\ 0)$

Gradients et mise à jour

$\partial J/\partial W^{(2)} = (-0{,}205;\ 0)$, $\partial J/\partial W^{(1)} = \begin{bmatrix} -0{,}205 & 0 \\ 0 & 0 \end{bmatrix}$

$W^{(2)} = (0{,}6205;\ -0{,}1)$, $w_{11}^{(1)} = 0{,}5205$, $b^{(2)} = 0{,}334 \Rightarrow \hat{y}$ augmente.

Initialisation

Symétrie

Poids tous égaux $\Rightarrow$ mêmes gradients $\Rightarrow$ neurones identiques pour toujours.

Initialisations aléatoires adaptées

- **He** : $\mathcal{N}(0,\, 2/n_{\text{in}})$ pour ReLU
- **Glorot (Xavier)** : variance $2/(n_{\text{in}} + n_{\text{out}})$ pour tanh/sigmoïde

Objectif : conserver la variance des activations et des gradients d'une couche à l'autre.

Plan

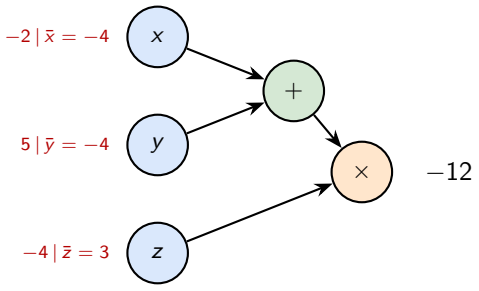
1. Le cycle d'apprentissage

2. L'algorithme de rétropropagation

3. Graphes de calcul

4. Différentiation automatique

Un graphe élémentaire : $f = (x + y)\, z$



Principe adjoint

$$\bar{v} = \frac{\partial f}{\partial v} = \sum_{u \in \mathrm{succ}(v)} \bar{u}\, \frac{\partial u}{\partial v}$$

Addition	distribue le gradient
Produit	échange les entrées
Max	aiguille vers l'argmax
$U = WA$	$\bar{W} = \bar{U} A^T$, $\bar{A} = W^T \bar{U}$

Plan

1. Le cycle d'apprentissage

2. L'algorithme de rétropropagation

3. Graphes de calcul

4. Différentiation automatique

Trois façons de dériver

Différences finies

$\frac{f(x+h)-f(x-h)}{2h}$
approchée ; 2 évaluations *par paramètre*

Symbolique

calcul formel ; exacte mais expressions qui explosent

Automatique

règle de la chaîne sur les opérations exécutées ; exacte et efficace

Modes de la différentiation automatique

Direct : une passe par *entrée*. **Inverse** : une passe par *sortie*.
Réseau : 10^6 paramètres, 1 perte $\Rightarrow$ mode inverse = **rétropropagation**, coût $\approx$ 2 à 3 forwards (mais mémoire des activations).

Mode direct pour $f = x^2 \sin x$ en $x = 1$

$(1, 1) \to u = (1, 2) \to v = (0{,}841;\ 0{,}540) \to f = (0{,}841;\ 2{,}223)$

GradientTape et boucle personnalisée

```python
x, y, z = tf.Variable(-2.0), tf.Variable(5.0), tf.Variable(-4.0)
with tf.GradientTape() as tape:
    f = (x + y) * z
print(tape.gradient(f, [x, y, z])) # [-4, -4, 3]

@tf.function
def train_step(x, y):
    with tf.GradientTape() as tape:
        loss = loss_fn(y, model(x, training=True)) # forward + coût
    grads = tape.gradient(loss, model.trainable_variables) # backward
    opt.apply_gradients(zip(grads, model.trainable_variables)) # mise à jour
    return loss
```

Vérification du gradient

Erreur relative $|g_a - g_n| / \max(|g_a|, |g_n|) < 10^{-6}$ avec $h \approx 10^{-5}$. Une conversion NumPy dans la bande coupe le graphe : gradient `None`.

À retenir

- $\boldsymbol{\delta}^{(L)} = \hat{\mathbf{y}} - \mathbf{y}$; $\boldsymbol{\delta}^{(l)} = \left(W^{(l+1)}\right)^T \boldsymbol{\delta}^{(l+1)} \odot g'(\mathbf{z}^{(l)})$; $\nabla_{W^{(l)}} J = \boldsymbol{\delta}^{(l)} \mathbf{a}^{(l-1)\,T}$.
- La rétropropagation est la différentiation automatique en **mode inverse**.
- Chaque nœud n'a besoin que de sa dérivée locale.

Questions de révision

1. Refaire l'exemple 2–2–1 avec $\mathbf{x} = (0, 1)$. ($\hat{y} \approx 0{,}570$)
2. Gradients de $\sigma(wx + b)$ pour $w = 2$, $x = -1$, $b = 3$.
3. Mode direct ou inverse pour $f : \mathbb{R} \to \mathbb{R}^{1000}$?